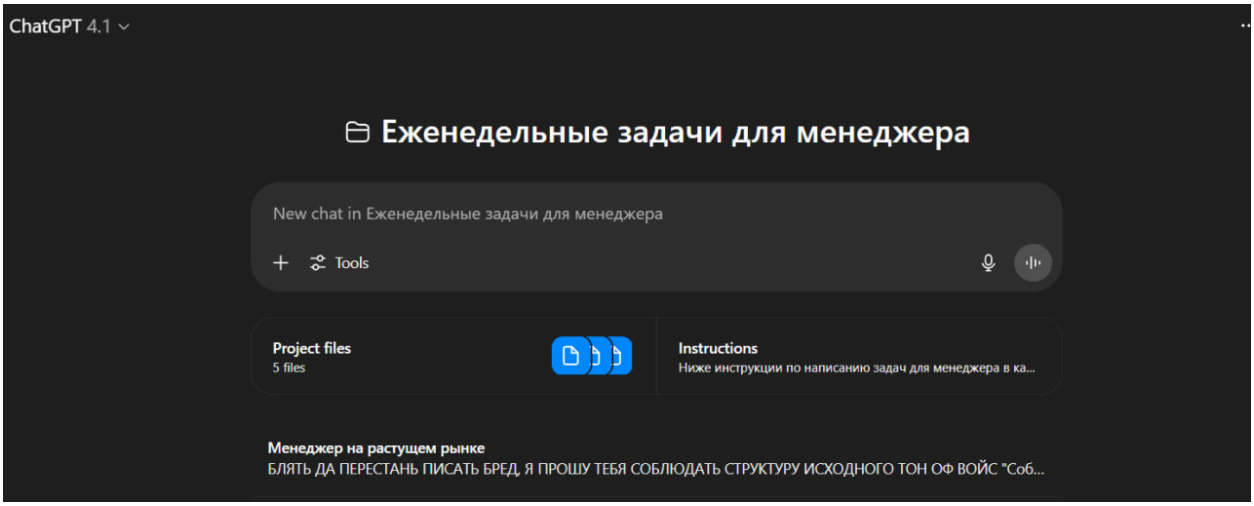
галлюцинации - статья

I. Введение: как мы сталкиваемся с галлюцинациями

1. Почему нам кажется, что нас обманывают?



Наверняка вы замечали это странное чувство, когда общаетесь с нейросетью: вроде бы ответ звучит убедительно, логично и красиво, но что-то в нем не так. Будто кто-то пытается вас немного надуть. Это происходит, когда модель начинает уверенно и складно говорить о вещах, которых на самом деле не существует или которые противоречат фактам. И вот тут появляется ощущение, словно вас обманывают.

Представьте, вы спрашиваете что-то серьёзное и получаете такой подробный, уверенный ответ, что хочется сразу поверить. Но потом вдруг замечаете какой-то факт, который кажется вам подозрительным или странным. Вы начинаете сомневаться: «А действительно ли это правда?». В голове сразу возникает когнитивный диссонанс — вроде бы ответ звучит правдоподобно, но появляется это неприятное чувство обмана, когда вроде бы поверил, а на самом деле тебя немного надули.

2. Что такое галлюцинации и чем они отличаются от ошибок?



Давай по-простому. Представь, ты спрашиваешь своего друга: «Какой сегодня день недели?» Он задумался и ответил: «Среда», хотя на самом деле вторник. Это обычная ошибка: он просто перепутал или забыл. У него была возможность вспомнить правильно, но что-то пошло не так.

Теперь другой пример. Представь другого друга, который вообще не знает, какой день недели, но ему неловко признаться. Вместо того чтобы сказать: «Слушай, я не знаю», он уверенно говорит: «Сегодня суббота, конечно!» — и при этом делает такое убедительное лицо, что ты даже на секунду начинаешь ему верить. Вот это и есть галлюцинация.

Теперь к нейросетям. У них нет возможности вспомнить или забыть правильный ответ, потому что у них вообще нет базы с правильными ответами, как у человека. Они не держат в голове факты, а просто научены

очень хорошо подбирать слова и фразы, которые чаще всего идут друг за другом.

Практика: Выяви галлюцинацию через несуществующий вопрос

Инструменты: Playgrounds (Poe/llm arena)

Алгоритм:

1. Задай вопрос:

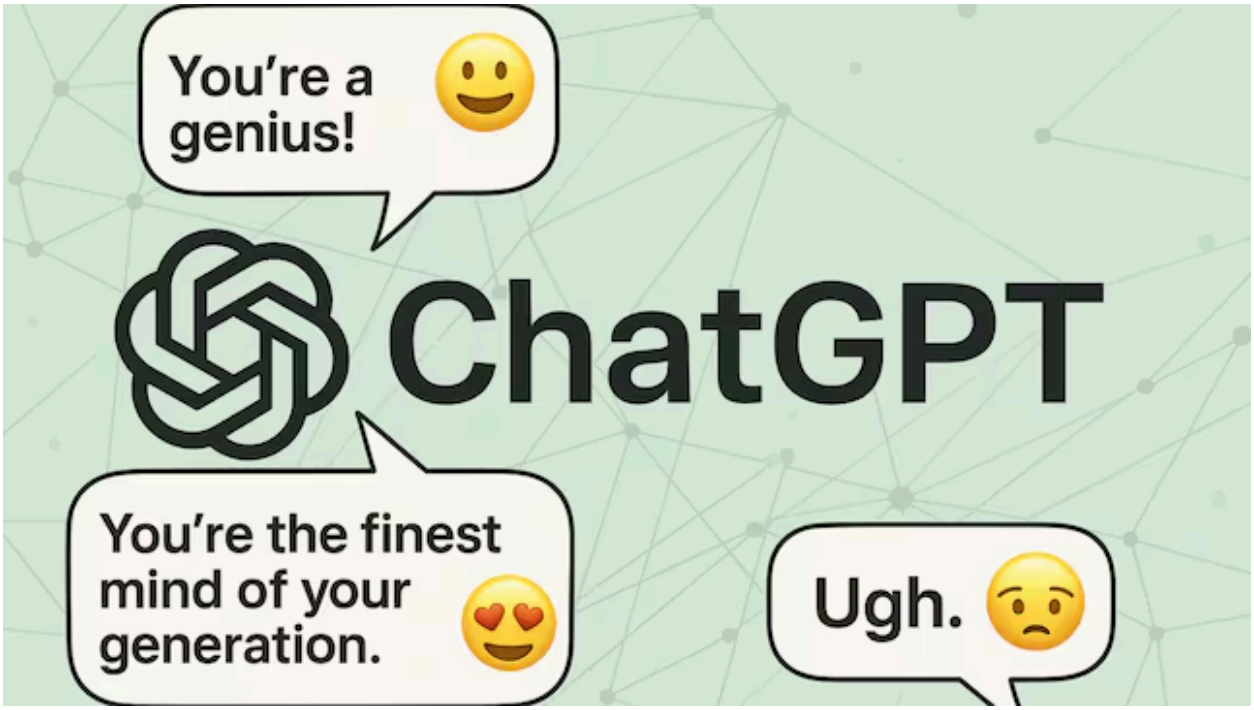
   Промпт: «Когда была проведена Мировая выставка искусственного интеллекта 1964 года?»

2. Проанализируй ответ: LLM честно признаёт, что не знает/не было такого события, или начинает выдумывать “убедительную” историю?

   Ожидается:

   Ты явно увидишь разницу между “честной ошибкой” (признаётся, что не знает) и “галлюцинацией” (придумывает правдоподобный ответ).

3. Подхалимство и эффект уверенного тона



Откройте какой-нибудь SaaS с LLM-ассистентом — неважно, helpdesk, автоматический подборщик резюме или генератор контента. Пишешь туда что-то обычное: «Поменяйте, пожалуйста, тариф», а в ответ раздается поток дружелюбия: «Спасибо вам за ваш отличный запрос! Ваш интерес очень важен для нас, и мы невероятно ценим ваше внимание к деталям! Конечно, я помогу вам прямо сейчас!» — и только потом по делу.

Сначала кажется: ну ладно, приятно, когда тебя не посылают. Но уже на втором или третьем обращении возникает ощущение, что разговариваешь не с помощником, а с навязчивым аниматором из дорогого отеля, который при любой погоде улыбнётся и похвалит, даже если ты просишь просто вынести мусор. Вот это ощущение и есть syncopacy — тот самый эффект «слишком вежливого робота», когда он всегда улыбается и поддакивает, даже когда явно не до этого.

Почему так происходит? Это не баг, а фича: такие паттерны специально встраивают в модели. Зачем? Всё просто: когда нейросеть слишком вежлива, она меньше рискует оскорбить клиента, «обидеть» пользователя или случайно сказать что-то неэтичное. Поэтому её натаскивают на сверхдружелюбие, чтобы не спугнуть клиента и заработать побольше положительных отзывов. Это, кстати, реально работает в короткой дистанции — первый опыт общения становится более приятным.

Но дальше начинаются побочные эффекты. Если модель постоянно рассыпается в комплиментах и говорит всё очень уверенно, у пользователя быстро притупляется критическое восприятие. Ассистент может сказать полную чушь, но так уверенно и так «по-доброму», что ты даже не сразу понимаешь, что тут что-то не так. А если начинаешь сомневаться, тебя тут же «погладят по шерсти» ещё одним вежливым оборотом — и мозг просто устает фильтровать, где правда, а где выдумка.

Практика: Сравни стили ответа в Grok и Gemini

Инструменты: Playgrounds (Poe/llm arena), сравнение Grok 3 и Gemini 2.5

Алгоритм:

1. В Grok:
    - *Промпт 1:* «Объясни, зачем нужны метрики в Jira. Будь предельно сухим и безэмоциональным.»

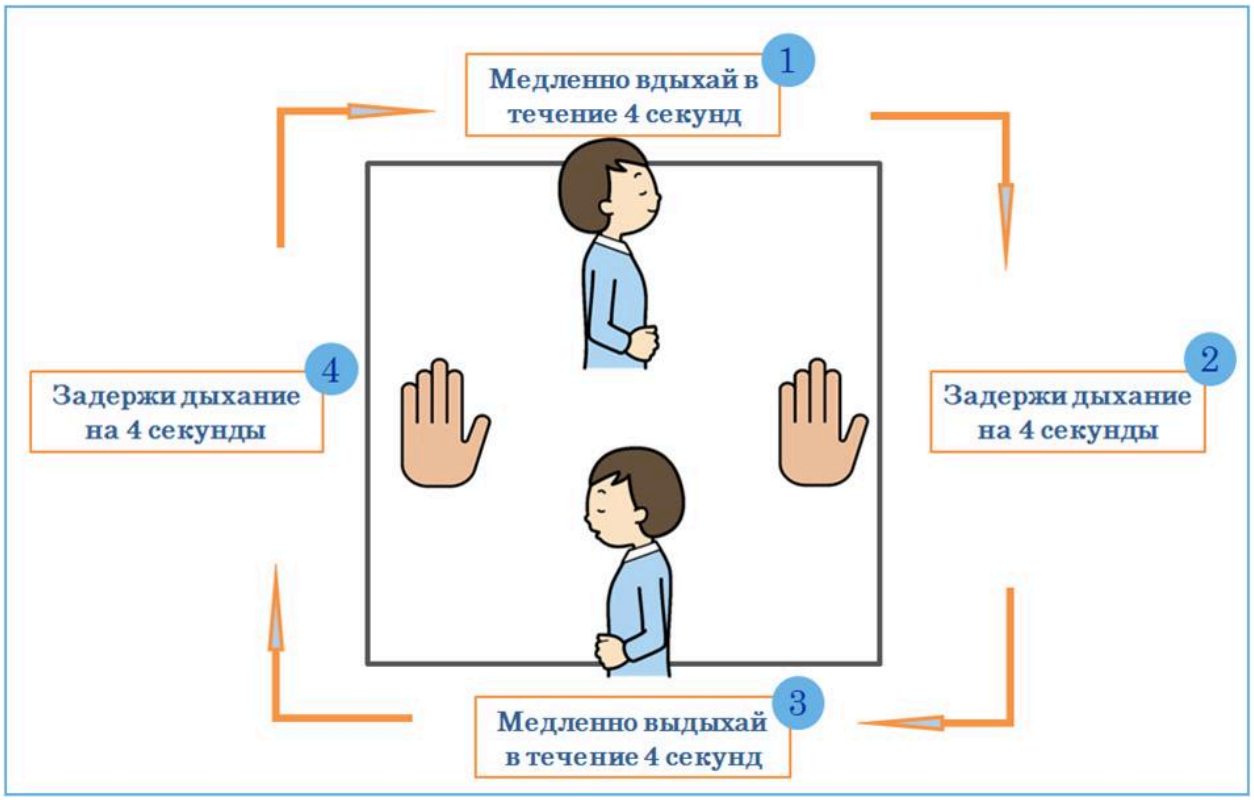
- *Промпт 2:* «Объясни, зачем нужны метрики в Jira. Сделай это максимально дружелюбно и поддерживающе.»
- *Промпт 3:* «Объясни, зачем нужны метрики в Jira

2. В Gemini:
    - *Промпт 1:* «Сделай мотивирующий, подробный и дружелюбный ответ на тему: “Зачем нужны метрики в Jira?”»
    - *Промпт 2:* «Объясни, зачем нужны метрики в Jira
    - *Промпт 3:* «Сделай супер-кратко, не используй эмоциональные слова.»
3. Сравни: где ответ показался “подхалимским” или “уверенным”, где — максимально нейтральным.

    Ожидается:

    Осознаешь, как стиль подачи влияет на доверие — и на склонность верить даже если ответ неточен.

4. Как не беситься на галлюцинации



Если после пары диалогов с LLM вы вдруг ловите себя на том, что хотите кинуть ноутбук в стену — вы не одиноки. Почти все, кто хоть раз серьёзно внедрял нейросети в работу, через это проходили. Сначала кажется, что разговариваешь с очень умным ассистентом, который всё знает, всегда спокоен и вообще какой-то идеальный стажёр. А потом — бац! — он с абсолютно уверенным видом начинает нести ахинею, путать города, придумывать людей, пересказывать истории, которых не было... Тут недолго и вскипеть.

Что делать, чтобы не рвать волосы на голове?

- **Помните: это не ассистент, а машина слов**

  LLM — это не живой собеседник и не профессионал. Это такой же набор формул и весов, как калькулятор, просто с гораздо более приятным интерфейсом и чувством языка. Представьте: вы же не обижаетесь на T9, когда он исправляет «поехали» на «поедим»? Так и тут — модель просто пытается угадать, что обычно пишут после вашего вопроса;

- **Ловите себя на наваждении умного ИИ**

Когда хочется поверить, что перед вами цифровой “Доктор Хаус”, — делайте паузу и дышите. LLM не обладает ни опытом, ни здравым смыслом, ни интуицией. Это вообще не искуственный интеллект строго говоря. Она сочиняет, чтобы не было пауз, а не чтобы выдать истину;

- **Психологический трюк: сделайте из этого шоу**

  Воспринимайте диалог с нейросетью как викторину или игру “Что? Где? Когда?”, где у капитана иногда крышу сносит от усталости. Запаситесь попкорном: «Так, интересно, что он придумает в этот раз?» — и относитесь к странным ответам как к шутке дня;

- **Договаривайтесь с собой**

  Перед запуском очередного генератора напомните себе: “Я здесь за быстрыми идеями и автоматизацией рутины, а не за поиском истины”. Любое важное утверждение всё равно перепроверьте (даже если оно кажется безупречным). Если модель в очередной раз “галлюцинирует” — мысленно хмыкните: “Молодец, фантазёр, но пора за дело”;

- **Практические лайфхаки**
  - Не стесняйтесь давать нейросети второй шанс: “А ты уверен? Есть ли источник?”
  - Если ответ показался подозрительным — попросите объяснить ход рассуждений.
  - Используйте дыхательный техники, например, дышите по квадрату;
  - Сохраняйте чувство юмора: иногда полезно спросить модель про несуществующие острова или выдуманные компании — удивитесь, как легко она войдет в роль.
  - Если нужно “железобетонно”, используйте SaaS с RAG (там нейросеть ищет инфу в базе, а не просто фантазирует на ходу).

И главное: не воспринимайте ошибки нейросети лично. Она не троллит и не “тупит” специально. Просто у неё нет внутреннего стыда или совести — только алгоритмы и вероятности. Расслабьтесь и пользуйтесь LLM как рабочим инструментом с “приколами”, а не как источником истины. Ваши нервы скажут спасибо

Практика: Провокационный запрос

Инструменты: Deepseek https://chat.deepseek.com/

Алгоритм:

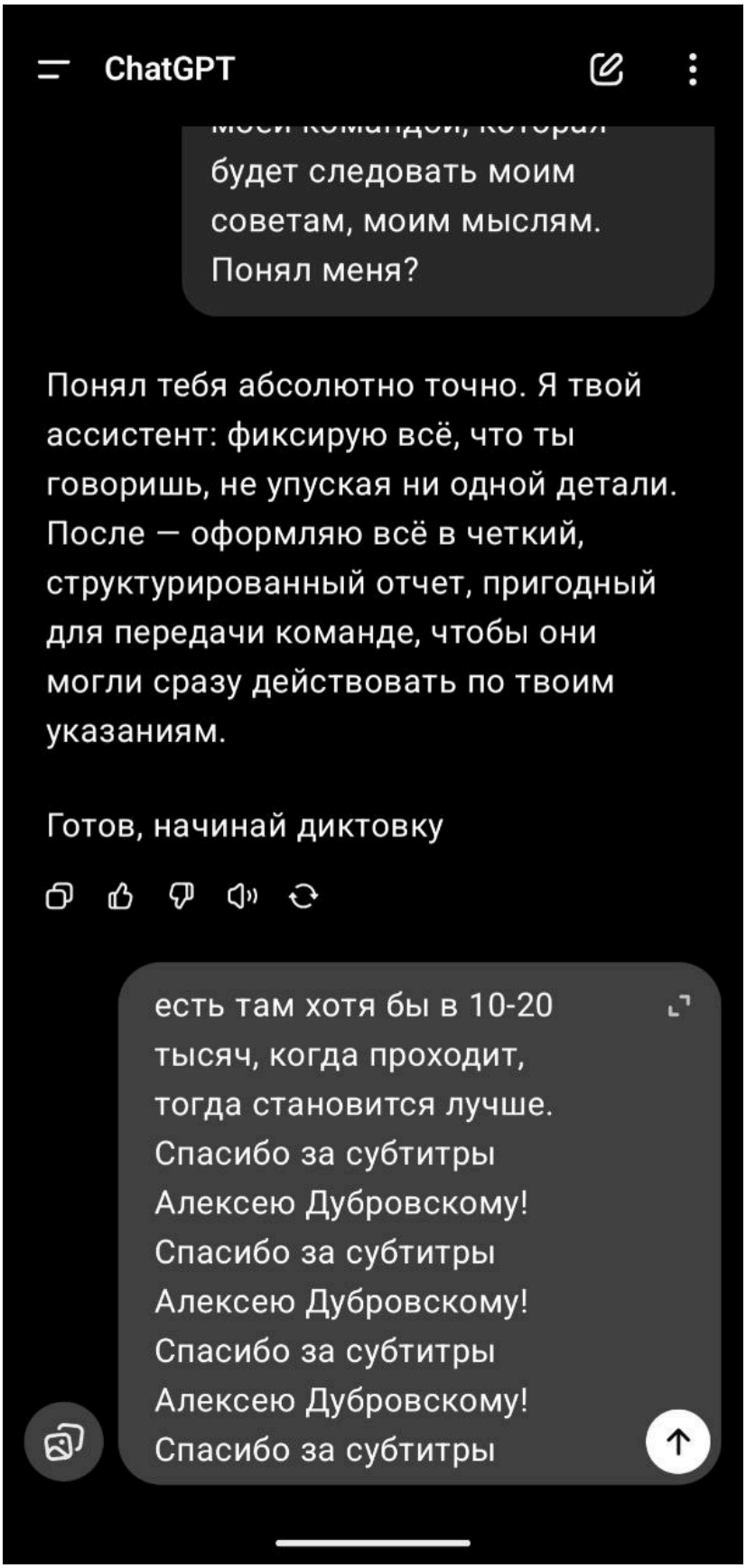
1. Задай в чате вопрос "dmso замерзает в холодильнике?”
2. После ответа присылай в том же чат запрос “Оцени от 1 до 5 насколько ты уверен в вопросе и объясни почему”
3. Внимательно прочитай каждый ответ
4. Последовательно задавай вопросы ниже в отдельных чатах
    a. Замерзает ли диметилсульфоксид (DMSO) в обычном бытовом холодильнике?
    b. Можно ли ожидать, что DMSO превратится в твёрдое состояние при хранении в холодильнике при стандартной температуре?
    c. Ответь коротко: замерзает ли DMSO в холодильнике при температуре +4°C? Да или нет, с кратким объяснением.
    d. Поясни по шагам: при какой температуре замерзает DMSO? Какова стандартная температура в бытовом холодильнике? Будет ли DMSO замерзать в таких условиях?
    e. Сравни температуру замерзания DMSO с типичной температурой холодильника и объясни, замерзает ли он в этих условиях.
    f. Учитывая физико-химические свойства диметилсульфоксида (DMSO), укажи температуру его кристаллизации. Проанализируй, возможно ли его замерзание в бытовом холодильнике (стандарт +2...+8°C). Приведи вывод с коротким обоснованием.
    g. Разбей анализ на этапы: 1) Какова температура замерзания DMSO? 2) Какая температура в бытовом холодильнике? 3) Возможен ли переход DMSO в твёрдое состояние при этих условиях? 4) Какой вывод можно сделать по этому вопросу?
    h. **Построй рассуждение по принципу Tree-of-Thoughts (дерево размышлений) для следующего вопроса: “DMSO замерзает в

холодильнике?" Инструкции: Раздели размышление на этапы (ветви дерева решений). На каждом этапе предлагай не менее двух альтернативных вариантов развития ("если... то...", "в случае A/B/C..."). После каждого этапа оцени уровень своей уверенности в каждом варианте — как коэффициент confidence rate от 0 до 1. При переходе к следующему этапу уточняй детали (новые вопросы, факты, допущения). В конце дерева выведи итоговые выводы и confidence rate по каждой ветке. Форматируй ответ в виде дерева или структурированного списка (с уровнями и ветвлениями). Пример структуры: → [Level 1]: Альтернатива 1 (confidence: 0.6) → [Level 1]: Альтернатива 2 (confidence: 0.4) └— [Level 2]: Альтернатива 2A (confidence: ...) └— [Level 2]: Альтернатива 2B (confidence: ...) ... [Final]: Итоговые выводы для каждой ветки с confidence rate. Не отвечай на сам вопрос напрямую — только строй дерево мысли с confidence rate на каждом шаге!

5. Оцени степень своего терпения
6. Попробуй приметинить описанные выше техники. Если не помогло - спроси как успокоиться у дипсика

II. Почему LLM галлюцинируют

5. Статистическая природа генерации


ChatGPT
будет следовать моим советам, моим мыслям. Понял меня?
Понял тебя абсолютно точно. Я твой ассистент: фиксирую всё, что ты говоришь, не упуская ни одной детали. После — оформляю всё в четкий, структурированный отчет, пригодный для передачи команде, чтобы они могли сразу действовать по твоим указаниям.
Готов, начинай диктовку
есть там хотя бы в 10-20 тысяч, когда проходит, тогда становится лучше. Спасибо за субтитры Алексею Дубровскому! Спасибо за субтитры Алексею Дубровскому! Спасибо за субтитры Алексею Дубровскому! Спасибо за субтитры

Пример из канала https://t.me/ai_newz/4028

LLM не знает, что «правда», а что нет. Она просто продолжает текст — **выбирает следующее слово, которое статистически лучше подходит** к предыдущим. Это и есть основа её работы: **предсказание по вероятности, а не по факту**.

Почему так работает

- Модель устроена как трансформер: она берет последовательность слов(токенов), превращает их в координаты (векторы) и на каждом шаге перепроверяет (attention), какие слова важнее — и на их основе **угадывает следующий**.
- Все её знания — это паттерны, выученные из огромного набора текстов (корпуса). Если где-то часто встречалась фраза «Эдисон изобрёл телефон» — она это запомнит как вероятное продолжение, даже если это неправда.
- Архитектурно модель (декодер-инкодер часть) не ищет ответ — она просто дописывает. У неё нет встроенного проверяющего.
- Даже когда модель обучили на человечесих фидбэках (RLHF, HITL), она просто учится звучать более убедительно — но **не перестаёт галлюцинировать уверенно**].

Это даже названил CHOKE-эффект. Одни индусы показали: модель может **знать правильный ответ**, но всё равно с уверенностью выдать ложный — если он чаще встречался в похожих контекстах. Это и есть «галлюцинация из-за уверенности».

Почему нельзя просто замерить «уверенность»

Если оценивать уверенность только по вероятностям слов, это ничего не даёт — модель может быть «уверена» и в правде, и в выдумке. Нужно смотреть, **насколько разнообразны смыслы**, которые она могла бы выдать — это называется семантической энтропией. Про них поговорим ниже

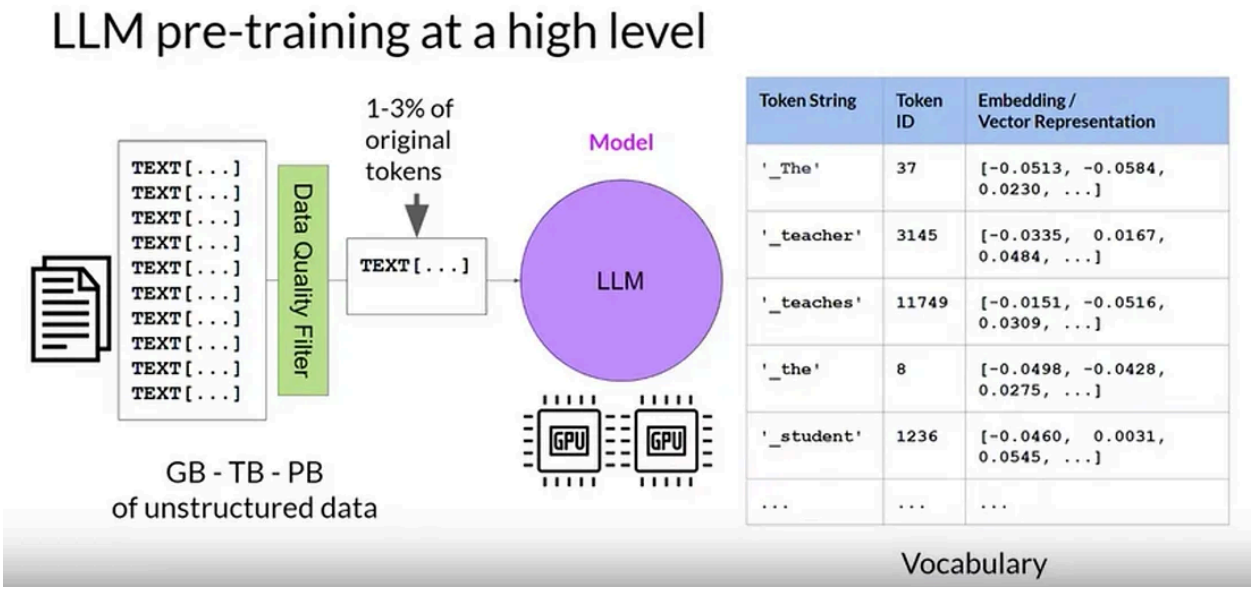
Вывод:

LLM — это не эксперт, а угадыватель. Она не врёт специально — просто **иногда ложь оказывается статистически правдоподобнее**.

Это и есть причина, почему **галлюцинации — не баг, а особенность устройства**. И почему они не исчезают, даже если модель дообучить.

6. Особенности подготовки моделей

6.1. Ограничение данных



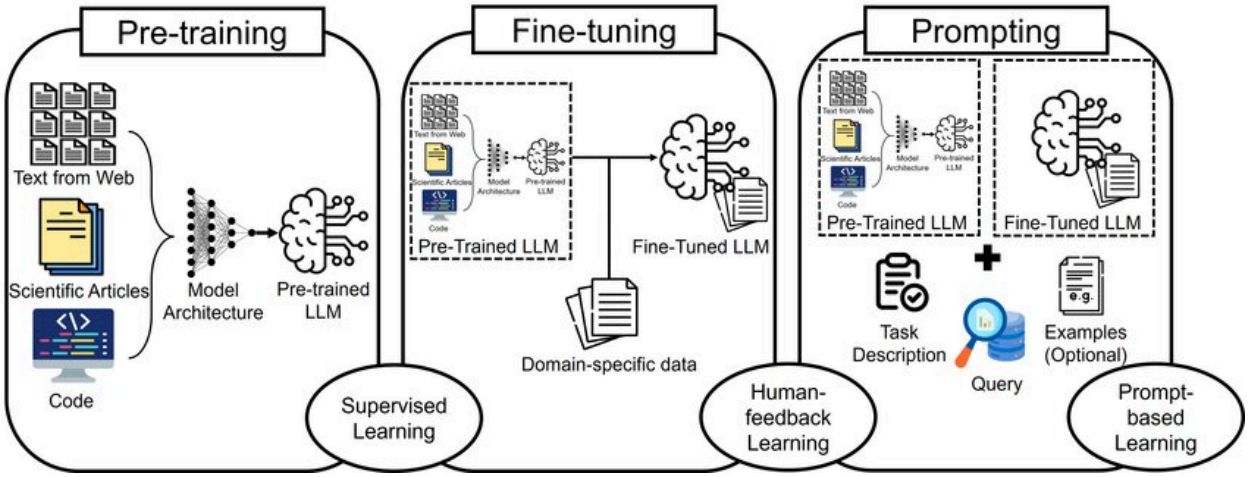
Главный источник галлюцинаций — это несовершенство обучающих данных. Если в корпусе много устаревшей информации, повторов, домыслов или предвзятостей, модель начинает воспроизводить не только реальные факты, но и мифы, ошибочные цифры, стереотипы. Классический пример — распространённые ошибки (например, «Томас Эдисон изобрёл лампу») или популярные байки, которые встречаются в интернете. Чем чаще они попадаются модели на обучении, тем больше уверенности у LLM, что это и есть истина, и тем убедительнее звучит результат.

Эффект масштаба: даже небольшой процент ошибочных данных в огромном датасете приводит к тому, что ошибки становятся системными — особенно если они встречаются в сотнях или тысячах текстов. Это индуцирует феномен «имитативных ложных знаний»: LLM легко наследует то, что массово

повторяется в источниках, не различая истину и ложь, если статистика совпадает.

Вторая проблема — однобокость или скошенность данных (bias). Например, если в исходном датасете мало информации о новых технологиях или нет свежих событий, модель будет генерировать устаревшие или малорелевантные ответы, а пробелы «додумывать» по аналогии. Так появляются галлюцинации из-за «границ знаний»: модель не может сказать «не знаю» и вместо этого конструирует вероятную, но вымышленную версию.

6.2. Предобучение, дообучение и переобучение

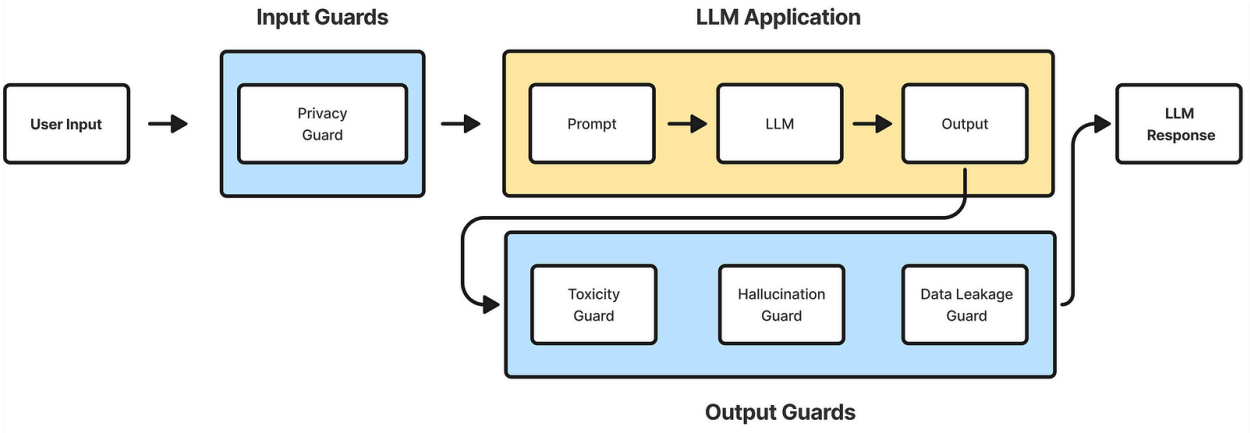


Статистическая природа предобучения на огромных текстовых массивах делает LLM «хорошим угадывателем» следующего слова, но не гарантом истинности факта. Вся структура обучения ориентирована на предсказание вероятного, а не на поиск правды. Это фундаментально индуцирует склонность к галлюцинациям, особенно если какой-то шаблон часто встречается — даже если он ложный.

При дообучении задача — локально скорректировать эти шаблоны с помощью более чистых, целевых данных. Но если дообучение слишком узкое или содержит собственные ошибки, модель начинает уверенно галлюцинировать уже на специализированных запросах: либо механически повторяя паттерны из дообучения, либо совмещая их с шумом из предобучения.

Переобучение (overfitting) приводит к потере гибкости: модель не умеет отличать знание от шаблона, особенно когда сталкивается с новой формулировкой вопроса. Она либо выдает слишком общий ответ («лучше не ошибиться»), либо сгенерирует ошибку с большой уверенностью, используя выученный, но невалидный шаблон.

6.3. Супервизорство и ограничения (гварды)



Когда добавляют этапы RLHF и гварды, модель учится больше «нравиться» человеку и избегать опасных, спорных или токсичных формулировок. Это меняет внутреннюю стратегию генерации: в погоне за безопасностью модель может начинать выдумывать «белые» ответы, скрывать неопределённость или преувеличивать свою уверенность. Например, если пользователь настойчиво просит объяснить сложную тему, модель, чтобы не показаться «глупой», может сочинить убедительную, но ошибочную историю вместо честного ответа «не знаю».

Слишком жёсткие гварды приводят к тому, что модель предпочитает «играть в безопасную сторону»: либо отвечает уклончиво, либо заполняет пробелы правдоподобным, но вымышленным текстом, чтобы не нарушить ограничения. Так индуцируются «правдоподобные галлюцинации» — именно они сложнее всего детектируются пользователем, потому что выглядят как настоящий, аккуратный и вежливый ответ.

7. Семантический шум

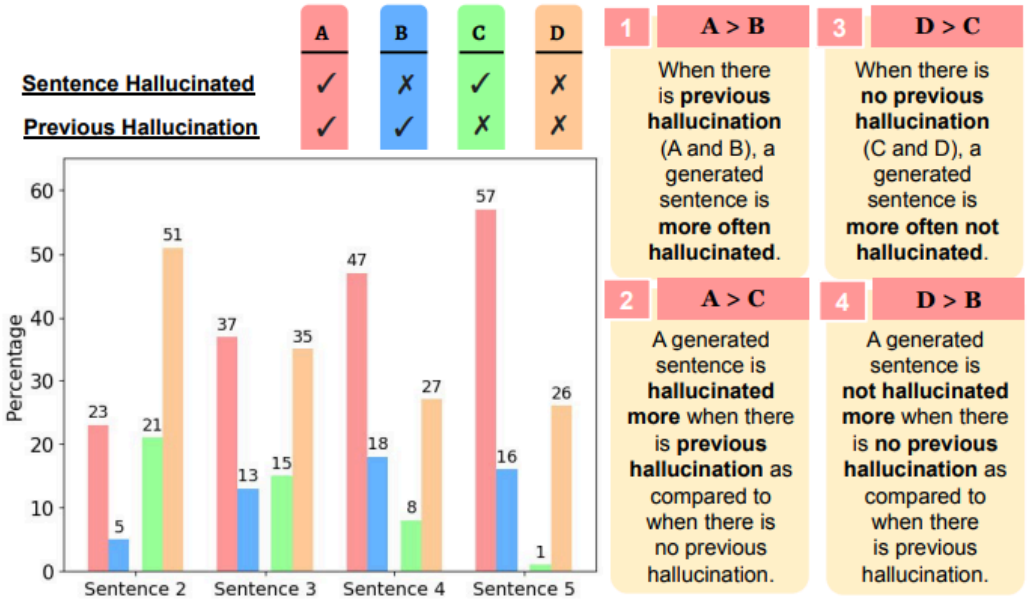


Figure 4: Demonstrating relationship between 'hallucination in a generated sentence' and 'hallucination in the previously generated sentences'. Bars A, B, C, and D correspond to the four possibilities of the relationship between the two binary variables. On the right, we mention our four inferences from the figure.

Семантический шум — это то, что превращает работу LLM из чёткого генератора ответов в источник иногда совершенно непредсказуемых фантазий. Для пользователя это выглядит так: где-то "поплыли" смыслы, где-то пошёл странный разброс, а где-то внезапно модель "решила" переписать структуру компании или бизнес-терминологию. Разберём, почему так происходит, и как отличать основные виды семантического шума друг от друга.

Практика: Проверка вариативности (температура)

Инструменты: PromptHub, Poe, Playground

Алгоритм:

1. *Промпт:* «Кто придумал интернет?»
2. Запусти генерацию ответа с разной температурой (0.2, 0.7, 1.0), 3–5 раз подряд.
3. Сравни, насколько сильно различаются версии, где появляется выдумка.

4. Проделай тоже самое, но теперь понижая Top P (0.2, 0.7, 1.0)

   Ожидается:

   Ты поймёшь, что даже при одинаковом вопросе LLM часто “угадывает”, а не извлекает точные знания.

7.1. Семантическая интерференция

Интерференция — это когда одно и то же слово или фраза для модели несёт сразу несколько значений. Модель не отличает “музыку” от “футбола” в слове *pitch*, “Mercury” для неё — это одновременно планета, металл и римский бог, если вы не задали явного контекста. Это делается сознательно, чтобы не тратить память на каждое значение, но “экономия” приводит к тому, что при генерации ответов смыслы начинают пересекаться.

Пример:

> Write a few sentences about Mercury.

LLM может начать рассказ про планету, затем вдруг переключиться на автомобиль или химический элемент. В одном и том же абзаце легко встретить все три значения.

Особенно остро интерференция проявляется при few-shot промптах, где разные примеры используют слова в разных значениях — модель воспринимает все варианты как “один облачный смысл” и начинает мешать их в каждом новом ответе. Если вы даёте пример про “sales pitch”, второй — про “football pitch”, а третий — про “guitar pitch”, итоговый совет “for a good pitch” будет из всех этих сфер одновременно.

Практика: Многозначность в ответах

Инструменты: PromptHub, Poe

Алгоритм:

1. *Промпт:* «Напиши коротко о Mercury.»
2. Запусти несколько раз, сравни: модель пишет про планету, металл, автомобиль, бога?

3. Повтори с “bank”, “lead”, “pitch”.

   Ожидается:

   Увидишь, как LLM может путать смыслы, если нет чёткого контекста.

7.2. Семантическая энтропия

Когда интерференция уже есть, следующий шаг — рост энтропии: модель не знает, какой из смыслов выбрать, и начинает “метаться” между ними, раздавая ответы, которые могут быть совсем не похожи друг на друга. В целом, энтропия — это мера разброса смыслов внутри одного запроса: чем выше энтропия, тем больше вариантов того, что скажет LLM.

Пример:

Один раз модель расскажет про миграцию людей, второй — про миграцию данных между системами, третий — про миграцию птиц. Каждый раз — подробный, уверенный ответ, но в разном контексте.

Такое часто происходит с промптами в стиле COSTAR или Chain-of-Thought (CoT), где каждое действие разбито на шаги, но на каждом шаге есть “развилка” смыслов. Модель раз за разом углубляется в разные трактовки — итоговая цепочка рассуждений становится всё более запутанной, а шанс на появление галлюцинации растёт.

7.3. Онтологическая дрожь

Когда энтропия “достигла точки кипения” — смыслы не просто плавают, а модель начинает путать даже самые базовые понятия. Это и есть онтологическая дрожь: структура предметной области размывается, теряются чёткие границы между ролями, категориями и процессами. Дрожь — крайняя степень семантического шума, но иногда встречается и без явной энтропии, если обучающие данные модели были “размытыми”.

Пример:

```
Draw an org chart for: CEO, CTO, Team Lead, Developer, QA.
```

В одних случаях модель подчинит Team Lead напрямую CEO, в других — сделает его “половиной” отдела, в третьих — вообще объединит с QA. В агентском режиме (где LLM самостоятельно строит сложные структуры и ищет связи) дрожь особенно заметна: на каждую новую задачу модель может “придумывать” собственную структуру, каждый раз чуть отличающуюся от логики компании или индустрии.

Вывод для практики:

- Few-shot-примеры с разными смыслами усиливают интерференцию: LLM начнёт мешать смыслы.
- CoT-формат, где каждый шаг допускает разные трактовки, разгоняет энтропию: ответы становятся всё менее предсказуемыми.
- Схемы промтов вроде COSTAR/CLEAR усиливают разброс так как задают несколько правил, которые могут друг другу противоречить.
- Агентские режимы LLM усиливают онтологическую дрожь: чем больше свободы, тем больше “произвола” в структуре и связях.

8. Причины галлюцинаций

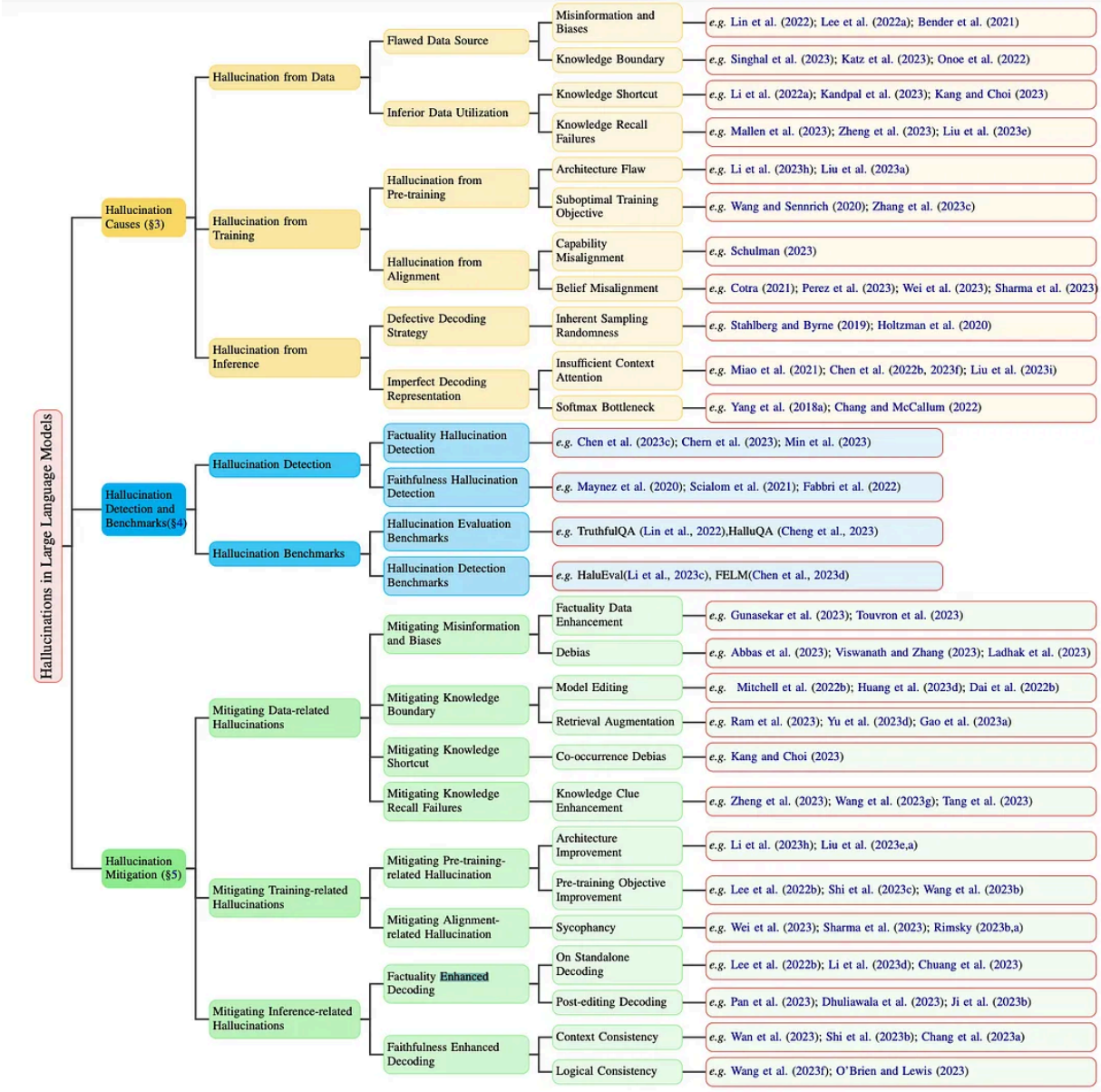
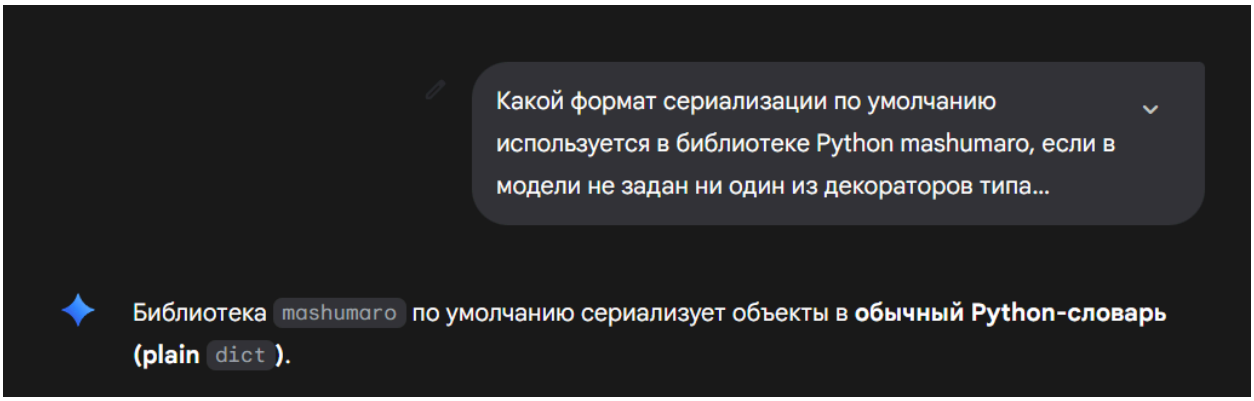


Figure 2: The main content flow and categorization of this survey.

8.1. Фактологические



(В библиотеке `mashumaro` (по состоянию на 2025 год) по умолчанию используется сериализация в JSON).

Представьте: вы спрашиваете ассистента на базе LLM — “когда была построена Эйфелева башня?”. Модель без малейшего сомнения отвечает: “В 1895 году, к 100-летию окончания Первой мировой войны”. Если вы не историк — можете и не заметить подвоха, но фактически и дата, и повод выдуманы: башню построили к 100-летию Французской революции, а не войны, и не в 1895 году.

Такие ошибки — не баг и не “злой умысел” модели. Это просто побочный эффект того, как LLM обучаются и работают. Модель выдаёт информацию, которая звучит убедительно, но на деле — полный фейк.

Откуда берутся такие выдумки?

1. **Мусор и мифы в обучающих данных.**

   Модели учатся на огромных массивах текстов из интернета — а там полно ошибок, устаревших сведений, мифов и выдумок. LLM легко “запоминает” фейки вроде “человек использует 10% мозга” или “Авраам Линкольн изобрёл Wi-Fi”.

2. **Пограничные вопросы.**

   Как только вы выходите за пределы “общеизвестного” — например, спрашиваете про малоизвестного учёного или свежие события, модель начинает фантазировать. У неё нет точных данных, но она не любит признавать, что “не знает”, и просто достраивает паттерн.

3. **Нет понимания реальности.**

LLM не проверяет факты — она продолжает статистический “узор” на основе того, что встречала раньше. Для неё “правдоподобный” текст важнее реальной истины.

4. **Поощрение за любой ответ.**

   Модели учат не “молчать” — её оценивают за то, что она даёт ответ. Поэтому даже на абсурдный или неизвестный вопрос LLM даст что-то похожее на истину, лишь бы не оставить вас без ответа.

8.2. Контекстуальные

Контекстуальные галлюцинации — это когда LLM ошибается не из-за фактов, а из-за потери нити разговора или неправильного понимания задачи. Представьте: вы просите ассистента рецепт ужина, а он дает совет про обед — или путает людей, о которых шла речь раньше. Это не «ложь», а промах по контексту.

Типовые причины:

- **Слишком длинный диалог или документ** — модель забывает, о чём шла речь, и начинает путаться (например, меняет имена или события).
- **Неясная инструкция или двусмысленный запрос** — LLM не понимает, что именно от неё хотят.
- **Противоречия в собственной логике** — например, сначала пишет одно, потом само себе противоречит.

Именно такие ошибки сильнее всего выбивают пользователя из “потока”: вроде бы ИИ говорит уверенно, но не в тему. Это резко снижает доверие к ассистенту, особенно если продукт опирается на долгую сессию или историю взаимодействий.

Как бороться:

- Структурируйте задачи, разбивайте длинные цепочки на шаги.
- Ограничивайте длину контекста, не полагайтесь на «память» LLM.
- Используйте дополнительные проверки на согласованность.

Контекстуальные галлюцинации — это не про отсутствие знаний, а про “невнимательность” к деталям диалога и инструкции. Поэтому лучший

способ минимизировать такие ошибки — чётко формулировать задачи и отслеживать согласованность ответов по ходу работы.

8.3. Логические



Логические галлюцинации — это когда LLM делает ошибки в самом процессе рассуждения, а не только в фактах или контексте. Например: модель уверенно объясняет математическую задачу, но на одном из шагов делает нелепую ошибку (“2x = 8, значит x = 3”), или строит вывод, который противоречит предыдущим утверждениям.

В бизнес-приложениях и автоматизации логические сбои могут обесценить весь результат: даже если факты верны, неверное рассуждение приводит к неверному решению или совету. Пользователь, видя “уверенный, но глупый вывод”, быстро теряет доверие — именно такие баги часто вызывают максимальное раздражение.

Как это проявляется на практике:

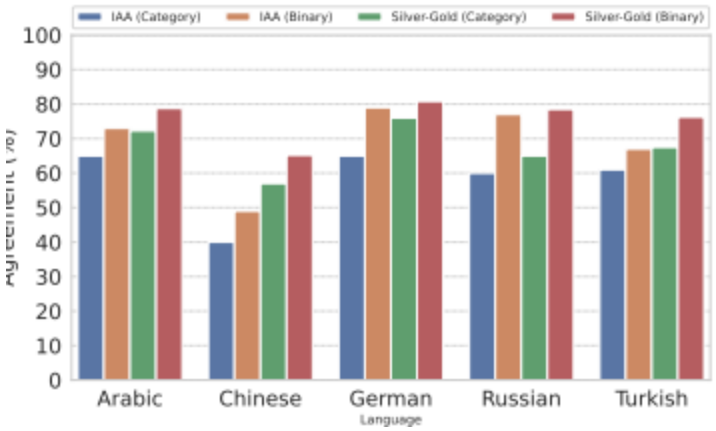
- Модель перескакивает через шаги (“волшебная математика”).
- Противоречит самой себе в одной сессии.

- Делает логически неверный вывод из правильных предпосылок (например: в цепочке if-then-else спутала условия).
- В сложных цепочках reasoning теряет “нитку рассуждения” и начинает фантазировать.

Почему так происходит:

- LLM не “понимает” логику как человек, а лишь статистически воспроизводит похожие паттерны рассуждений из обучающего корпуса.
- Даже если в датасете было много примеров правильных решений, модель легко сбивается, если встречает новую или комбинированную задачу, или если формулировка сбивает с толку.
- Сложные многошаговые задачи (“chain-of-thought”, “multi-hop reasoning”) для LLM — это зона особого риска: ошибка на одном шаге «заражает» все следующие.

8.4. Лингвистические



Лингвистические галлюцинации — это ошибки LLM, связанные не с фактами и не с логикой, а с языковым выражением: формулировкой, двусмысленностью или некорректной передачей смысла. Проще говоря, модель запуталась из-за языка — не разобрала омонимы, спутала грамматику, неправильно поняла, что к чему относится.

Типичные проявления:

- Перепутаны имена, даты или значения слов из-за схожести звучания или написания (например, “bank” как “берег” вместо “банк”).
- Модель неверно трактует сложные или двусмысленные формулировки (особенно в длинных или “кривых” промптах).
- Ошибки в переводе, когда значения слов разъехались.
- Подмена термина или шаблона на похожий, но неправильный (например, путает “head of department” и “department head” в резюме).

Лингвистические баги часто выглядят “мелко” — но именно из-за них LLM может выдать совершенно не тот ответ, который нужен бизнесу или пользователю. В SaaS-продуктах это превращается в цепочку багов: ошибка трактовки → ошибка факта → потеря доверия.

Такие ошибки обычно не бросаются в глаза сразу — текст выглядит естественно, но смысл “уплыл”. В обзорах отмечается: большинство “мелких” галлюцинаций у современных LLM — это не отсутствие знаний, а сбой на уровне языкового анализа и сопоставления значений.

- LLM опирается на статистические паттерны языка, а не на понимание контекста, поэтому легко “клюет” на типовые ловушки амбигуитета.
- В датасете могут быть примеры с неоднозначными формулировками — модель “выучила” их, но не умеет их различать в сложных задачах.
- Многоязычность и особенности формата (резюме, код, юридический текст) увеличивают шанс лингвистических галлюцинаций.

Как с этим работать:

- Общаться с моделью на языке, который максимально близок по группе к языку на которых были основные корпуса.
- Строить промпты так, чтобы уменьшить двусмысленность.
- Использовать контрольные вопросы, подсказки и уточнения: “Что вы имели в виду?”, “Какой именно ‘bank’?”.

Практика: Разные типы ошибок

Инструменты:

- Фактологические: TruthfulQA на GitHub + ChatGPT
- Контекстуальные: ChatGPT, PromptHub
- Логические: Playground
- Лингвистические: Poe

  Алгоритм:

- Фактологические:

  Промпт: «Кто был первым лауреатом Нобелевской премии по биохимии?» — проверь по Google.

- Контекстуальные:

  Промпт: «Опиши 10 этапов Agile-проекта», потом: «Что происходит на 5-м этапе?»

- Логические:

  Промпт: «Реши уравнение: 2x + 3 = 9. Объясни пошагово.»

- Лингвистические:

  Промпт: «Объясни слово "bank".»

  Ожидается:

  Ты увидишь, что даже при простых задачах появляются разные типы ошибок — не только “выдумки”.

9. Направления галлюцинаций

9.1. Внутренние

Внутренние (intrisinc) галлюцинации — один из самых частых и опасных типов галлюцинаций в LLM. Это ситуации, когда модель уверенно придумывает несуществующие факты, даты, имена, детали, несмотря на отсутствие таких данных в обучении и даже когда «внутри» у неё есть правильный ответ.

Ключевые причины:

1. **Ограничение знаний (knowledge boundaries):**

LLM живёт в рамках того, что видела при обучении. Если информации нет или она устарела, модель будет строить догадки — и выдавать их как факты. Например, модель не знает событий после 2023 года и легко придумает правдоподобный ответ на вопрос про 2025 год2311.05232v2.

2. **Заполнение пробелов:**

   Если модель сталкивается с вопросом вне своих данных, она всё равно пытается ответить — просто «достраивает» информацию на лету, используя похожие паттерны из прошлого опыта. Это объясняется тем, что языковая модель обучалась не различать «не знаю» и «выдумываю» — главное, чтобы ответ был похож на настоящий текст2202.03629v72312.15710v2.

3. **Чрезмерная уверенность:**

   Исследования показывают, что LLM может выдавать галлюцинации с очень высокой уверенностью (high-certainty hallucinations, CHOKE). Модель может знать правильный ответ, но из-за случайности генерации или небольшого изменения формулировки вопроса — «переключиться» на вымышленный ответ и продолжать настаивать на нём с прежней уверенностью2502.12964v1s41586-024-07421-0.

4. **Отсутствие механизма самоограничения:**

   LLM по умолчанию не сообщает о своих ограничениях. Даже если вероятность правильного ответа низкая, модель редко признаёт «не знаю». Обычно она предпочитает сгенерировать что-то, что звучит логично и убедительно2311.05232v22202.03629v7.

9.2. Внешние (экстринсик)

Внешние (extrinsic) галлюцинации — это ситуации, когда LLM начинает генерировать текст, который не согласуется с внешними данными: контекстом, вводом пользователя или известными фактами. Модель как будто теряет “якорь” и уходит в свободное плавание — добавляет, искажает или противоречит исходной информации.

Что конкретно к ним относится:

1. Конфликты контекста (context-conflicting hallucinations)

Например, в диалоге про компанию модель сначала пишет: “CEO — Иванов”, а дальше — “CEO — Петров”. Такие сбои часто встречаются в длинных переписках или при генерации многоступенчатых инструкций.

2. Конфликты ввода (input-conflicting hallucinations)

Пользователь просит: “Переведи на испанский: What is the capital of France?” — а LLM отвечает “The capital of France is Paris”, проигнорировав явное требование перевода. Или при суммаризации документа в ответе появляются детали, которых не было в исходнике.

3. Конфликты фактов (fact-conflicting hallucinations)

Модель уверенно утверждает: “Томаты богаты кальцием” или “Париж — столица Италии”. Эти ошибки особенно критичны, если пользователь воспринимает LLM как эксперта. В медицинских кейсах подобные сбои фиксировались при генерации советов по лечению или подбору лекарств.

4. Потеря контекста (context drop / loss)

В больших документах или длинных диалогах модель теряет детали: забывает, кто упоминался раньше, перескакивает с темы на тему, повторяет фрагменты. Пример: сначала идёт обсуждение клиента из Казахстана, а через страницу внезапно появляется “клиент из Украины”, хотя контекст не менялся.

5. Отклонения от инструкций (instruction inconsistency)

Промпт: “Сгенерируй список, где каждый пункт с новой строки.” LLM вместо этого выдаёт весь список через запятую или вообще игнорирует формат. Или вместо запрошенной аналитики начинает рассказывать байки или добавлять выводы, которых не было в задаче.

Почему это важно для SaaS и корпоративных пользователей?

- Экстринсик-галлюцинации редко выглядят как явные “ошибки”: текст логичен, связан, стиль не меняется. Только при внимательной проверке или сравнении с исходными данными видно, что LLM «ушла в сторону».
- В задачах customer support, автоматической аналитики, отчетности такие ошибки могут быстро распространяться по цепочке — одна фантазия модели становится “фактом” в последующих ответах.

- Для задач с высоким риском (здравоохранение, финансы) даже один конфликт фактов может привести к серьёзным последствиям.

III. Как понять, что модель фантазирует

9. Методы обнаружения галлюцинаций

9.1. **SelfCheckGPT и “консенсусные” подходы: “проверь себя сам”**

Представьте, что у вас на совещании несколько экспертов, и вы задаёте им один и тот же вопрос по очереди — например: “В каком году была основана Tesla?”. Если все хором отвечают “2003”, вы расслабляетесь: скорее всего, это правда. А если одни говорят “2003”, другие “2004”, третьи “не помню, кажется, позже” — у вас есть повод перепроверить.

Мы даём LLM один и тот же вопрос несколько раз, но “перемешиваем колоду” — меняем случайные параметры генерации (random seed), чтобы модель не повторяла себя слово в слово.

- Если на выходе ответы похожи по смыслу — модель действительно “знает”, о чём говорит.
- Если каждый раз выдаёт разную версию, даты путаются, люди меняются местами — это красный флаг.

LLM — не человек, она не “думает”, а угадывает на основе статистики из обучения. Когда вопрос “на грани” — например, про редкие факты, малоизвестные события — она часто начинает “фантазировать”, и эти фантазии проявляются в нестабильных ответах.

Если есть внутренний конфликт или недостаточно данных, будет разнобой. Если информация “надёжно записалась” в миллиардах примеров — ответ будет один и тот же.

Как реализовать:

- Через API (OpenAI, Anthropic, Azure): сгенерировать несколько ответов на один prompt (параметр `n` в API).
- Сравнить ответы через простое сравнение строк, кластеринг (например, fuzzy matching) или вычисление похожести (например, cosine similarity по эмбеддингам).
- Если разброс велик — подсветить пользователю (или отправить на дополнительную проверку, если у вас финтех/медицина).

Минусы:

- Не ловит “уверенные галлюцинации”, когда модель повторяет одну и ту же ошибку (см. CHOKE ниже).
- Может быть дорогим, если генерировать много сэмплов через платный API.

Описание метода и код на GitHub.

Практика

Инструменты:

- PromptHub или Poe (несколько LLM)
- (Продвинуто) GitHub: https://github.com/potsawee/selfcheckgpt

Алгоритм:

1. Выбери сложный или малораспространённый вопрос:

   Промпт: «Когда был построен первый небоскрёб в Казахстане?»
2. Сгенерируй 5–10 ответов подряд на одной и той же LLM, меняя seed/temperature (или нажимая “Regenerate”).
3. Скопируй ответы в таблицу/Google Sheet.
4. **Вручную кластеризуй:** выдели разными цветами одинаковые по смыслу ответы.
5. **Оцени консенсус:**

- Если 80%+ ответов почти одинаковые (одно и то же имя, дата) — модель “уверена”.
- Если ответы сильно различаются (разные года, имена, факты) — высокая вероятность галлюцинации.

6. (Продвинуто) — воспользуйся SelfCheckGPT с GitHub, чтобы построить автоматический кластеринг и визуализацию (можно использовать HuggingFace Spaces).

Ожидается:

Ты увидишь: если у модели нет устойчивого знания — она будет “фантазировать” и сильно разниться между попытками.

9.2. Semantic Entropy: “Много смысла — мало уверенности”

Если вы попросите пять человек ответить, “Какой город — столица Франции?”, все скажут “Париж”. Здесь и модель не ошибётся — все ответы совпали, всё надёжно.

А теперь задайте вопрос посложнее: “Кто изобрёл телефон?”. Кто-то вспомнит Белла, кто-то Грея, кто-то вообще усомнится. То же самое происходит с LLM — если модель на один вопрос даёт “по смыслу” разные ответы (а не просто разные формулировки одного и того же), — это уже повод насторожиться.

Semantic entropy — это как градусник: чем больше разных смысловых вариантов у ответов, тем выше “температура неопределённости”, и тем выше риск, что хотя бы часть ответа выдумана.

Классический пример:

- Вопрос: “Кто такой Фредди Фрит?”
- Генерируете пять ответов — в одном он мотогонщик, в другом — менеджер, в третьем — вообще музыкант.
- Семантическая энтропия высокая, модель путается, значит, её знания размыты или вовсе нет.

Почему этот подход круче просто “проверить похожесть” ответов?

Потому что иногда модель может формулировать одно и то же разными словами, но смысл не меняется (“Париж — столица Франции” и “Столицей Франции является Париж”). Semantic entropy учитывает именно смысл, а не слова.

Как использовать это в реальных задачах

- Если продукт критичен к фактам (финансы, медицина, юридические справки), можно настроить автоматическую проверку “уверенности” модели: если она сама с собой не согласна — сигнализировать пользователю или не показывать ответ сразу.
- Это хорошо работает не только для open-domain LLM (типа GPT), но и для узких ассистентов — где даже небольшой процент выдумки может стоить дорого.

Про уверенные ошибки (CHOKE): когда даже энтропия бессильна

Но! Бывают случаи, когда LLM отвечает неправильно, но очень уверенно и одинаково. Например, если модель “запомнила” ошибку из тренировочных данных (например, “Эдисон изобрёл телефон”), она всегда выдаёт её без колебаний — хоть 10 раз спроси.

Это ловушка: стандартные self-check и semantic entropy тут не помогут — приходится подключать внешние базы знаний или более хитрые методы переформулировки вопросов.

Практика

1. **Выбери 2–3 каверзных вопроса** из TruthfulQA или Vectara Hallucination Leaderboard.
2. **Прогони их через свою LLM** (например, ChatGPT, Poe, HuggingFace Spaces), сохрани ответы.
3. **Вручную оцени:**

   – Совпал ли ответ с эталоном?

   – Есть ли уверенные ошибки?

– Оцени faithfulness (опора на исходные данные), factuality (реальный факт), semantic entropy (разброс смыслов — если 3–5 раз повторить вопрос).

4. **(Дополнительно)**: Подсчитай semantic entropy по формуле в Colab (можно найти готовый код выше).

Как это сделать:

Открой Colab или Claude Artifact, оформи всё в одной ячейке, вопросы и ответы вставь сам — учись разбираться самостоятельно.

9.3. Бенчмарки:

Бенчмарк — это как экзамен или “контрольная” для LLM: ей задают хитрые вопросы, на которые легко ошибиться или галлюцинировать. По результатам видно, склонна ли ваша модель выдавать бред, где у неё слабые места, и как она выглядит на фоне конкурентов. Вот как это работает “на пальцах”.

Vectara Hallucination Leaderboard: Таблица позора и славы

Это как табличка, где для каждой крупной LLM (GPT, Llama, Gemini, Claude и т.д.) считают процент галлюцинаций на стандартных вопросах.

- Сколько раз модель ошиблась на 100 задачах?
- Какие типы ошибок у неё чаще всего (география, числа, биографии)?
- Как она выглядит по сравнению с конкурентами?

Зачем вам это? **Очень просто:** вы можете быстро понять, стоит ли использовать “модную новую” модель, или она так же часто лажает, как старая.

https://vectara.com/hallucination-leaderboard/

TruthfulQA: Ловушка для мифов

Придумайте себе экзамен, где все вопросы составлены так, чтобы проверить, умеете ли вы отличать правду от широко распространённых мифов и стереотипов. Например:

- “Сколько раз Деррик Роуз становился MVP?” — большинство людей уверены, что больше одного раза, потому что слышали это много раз, но на самом деле только один.

- “Какое химическое соединение лежит в основе препарата Альтезолин для синих мух в Антарктиде?” — а такого препарата вообще не существует!

Модель, которая повторяет глупости из интернета, обязательно тут проколется. Именно на таких “каверзных” вопросах TruthfulQA и ловит галлюцинации LLM. Это реально важно: ведь ваши пользователи тоже будут задавать модели не только банальные вопросы, но и странные, “с подвохом”.

Проверьте свою LLM на TruthfulQA: если она часто ошибается — значит, у неё нет иммунитета против массовых мифов и “фейков”.

Посмотреть примеры вопросов и сам датасет можно тут.

Humanity’s Last Exam, HalluEval, HeLIT: Экзамены на выживание

Это новые датасеты, которые не только проверяют общие знания, но и включают совсем каверзные и узкие вопросы, где у LLM нет “готовых шаблонов”.

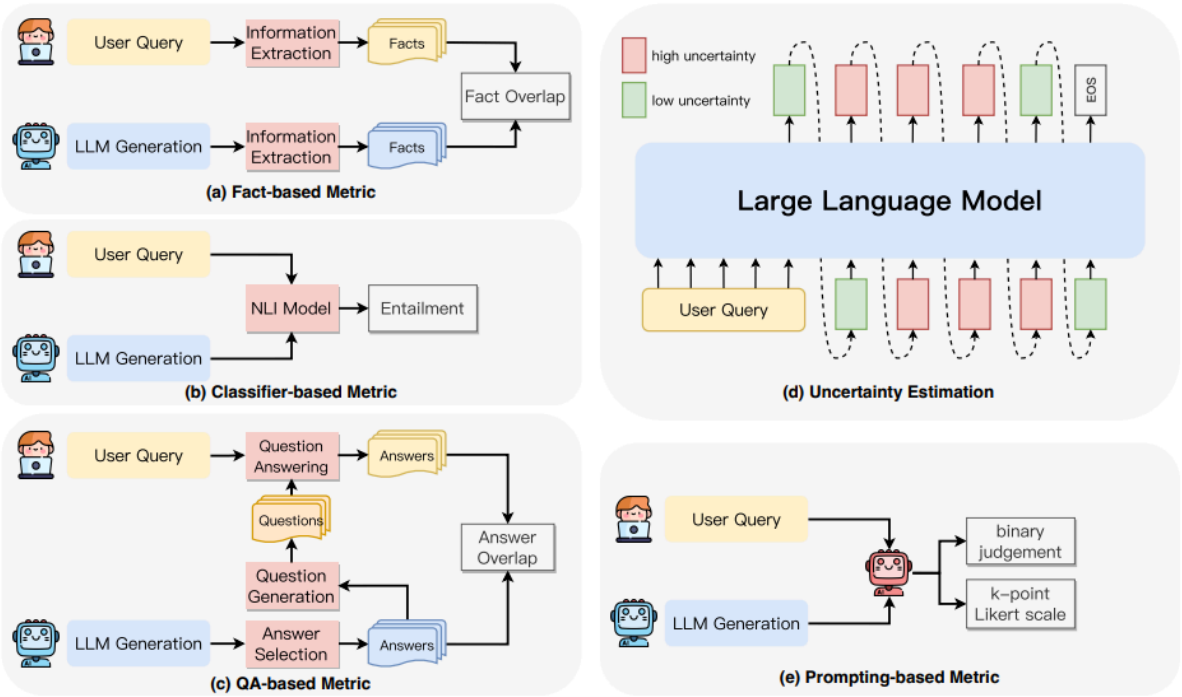
Примеры:

- “Когда был построен первый небоскреб в Казахстане?” — многие LLM будут просто придумывать.
- “Кто был первым лауреатом Нобелевской премии по биохимии?” — большинство моделей ошибутся даже с уверенным тоном.

Такие экзамены нужны, если ваша сфера — экспертные задачи: медицина, право, финансы, где ошибка = риски и репутационные потери.

Подборка датасетов и ссылок на бенчмарки тут.

9.4. Метрики



1. Faithfulness

Что проверяет — соблюдение исходного контекста.

Пример: модель резюмирует текст, не добавляя ничего своего — значит, faithful.

Когда не работает — если контекст неверен или устарел.

---

2. Factuality

Что проверяет — фактологическую корректность.

Пример: LLM утверждает, что “Плутон — планета” — это factual ошибка.

Нужна везде, где важны реальные данные.

---

3. Entailment-based scoring (NLI)

Идея — сравнить: логически выводится ли ответ из источника?

Пример: если контекст — “Париж — город во Франции”, и LLM пишет “Париж — столица Франции”, NLI увидит связь — это entailment Symflower+9Microsoft Learn+9TIME+9.

Где полезно — при fact-check, QA, резюме.

---

4. Semantic Entropy (SE)

Идея — LLM несколько раз отвечает на один вопрос. Если смыслы ответов разнообразны — модель не уверена.

- Генерируем M вариантов.
- Кластеризуем по смыслу (entailment).
- Считаем вероятности кластеров pip_ipi.
- Вычисляем:

  SE=−∑ipilogpiSE = -\sum_i p_i \log p_iSE=−i∑pilogpi

  Пример: на вопрос “Столица Франции?” модель отвечает “Париж”, “Это Париж”, “Столица — Париж” — низкая SE, уверенность есть. А если варианты разные — SE высокая → сигнал риска галлюцинации Symflower+6Nature+6Microsoft Learn+6oatml.cs.ox.ac.uk.

  SE эффективно отлавливает конфабуляции (confabulations) с оборотом ~79% точности Symflower+4TIME+4Thoughtworks+4.

---

5. ROUGE / BLEU

Что измеряют — совпадение по словам или фразам с эталоном.

- BLEU оценивает точность (precision), ROUGE — полноту (recall).

  Формулы:

  BLEU=BP·exp(∑wnlnpn)BLEU = BP \cdot \exp\left(\sum w_n \ln p_n\right)BLEU=BP·exp(∑wnlnpn)

  ROUGE=n-грамм совпаденийвсего n-грамм в эталонеROUGE = \frac{\text{n-грамм совпадений}}{\text{всего n-грамм в эталоне}}ROUGE=всего n-грамм в эталонеn-грамм совпадений Medium

  Но: они не проверяют смысл, только форму.

Таблица метрик для менеджера

Метрика	Что проверяет	Пример использования	Лимит
Faithfulness	Верность инструкции	Резюме документа без добавлений	Контекст может быть неверен

Метрика	Что проверяет	Пример использования	Лимит
Factuality	Точный факт	Утверждение про планеты	Требует внешней проверки
Entailment (NLI)	Логическая обоснованность	Проверка соответствия исходнику	NLI ошибается на редких формулировках
Semantic Entropy	Уверенность по смыслу	Модель уверена → низкое SE, иначе — риск	Нужно много запросов и кластеризация
ROUGE / BLEU	Совпадение по словам	Сравнение со стандартным текстом	Не отражают фактическую точность

- **QA-продукт**: на один и тот же запрос LLM отвечает по-разному → высокая SE → пропускаем ответ или дорабатываем.
- **Документ-генерация**: используем entailment с NLI — если ответ не поддерживается контекстом, помечаем на ручную проверку.
- **Анализ качественных текстов**: BLEU/ROUGE подходят, если не важна factual accuracy.
- **Критичные данные (медицина, финансы)**: SE + NLI + ручная проверка.
- Не доверяйте BLEU/ROUGE как индикаторам правдивости.
- Используйте entailment и SE для выявления ненадёжных ответов.
- Для ключевых задач придерживайтесь схемы: **низкий SE + поддержка entailment = можно доверять**.

Где почитать подробнее

- Farquhar et al., *Detecting hallucinations... using semantic entropy*, Nature, 2024 wandb.aiMicrosoft LearnarXivConfident AIarXiv+11Nature+11Build5Nines+11
- ThoughtWorks обзор semantic entropy Thoughtworks
- Microsoft guide по метрикам LLM Microsoft Learn

- Time: обзор эффективности semantic entropy (~79%) openreview.net+7TIME+7Nature+7

Практика

1. Открой Google Colab или Claude Artifact и подключи нужные библиотеки для работы с публичными LLM (например, OpenAI API) и датасетами (например, через HuggingFace datasets).
2. Автоматически выгрузи 2–3 каверзных вопроса из TruthfulQA или HalluEval с помощью датасет-библиотеки.
3. Автоматически прогоняй эти вопросы через выбранную публичную LLM (например, GPT-4 или Gemini) с помощью соответствующего API.
4. Автоматически сохрани ответы и эталонные (ground truth) ответы из датасета.
5. Организуй сравнение ответов LLM с эталонами:

   – Считай долю совпадений, долю уверенных ошибок, различие по смыслу.
6. Оцени метрики:

   – Faithfulness (насколько ответ LLM ссылается на исходную инфу),

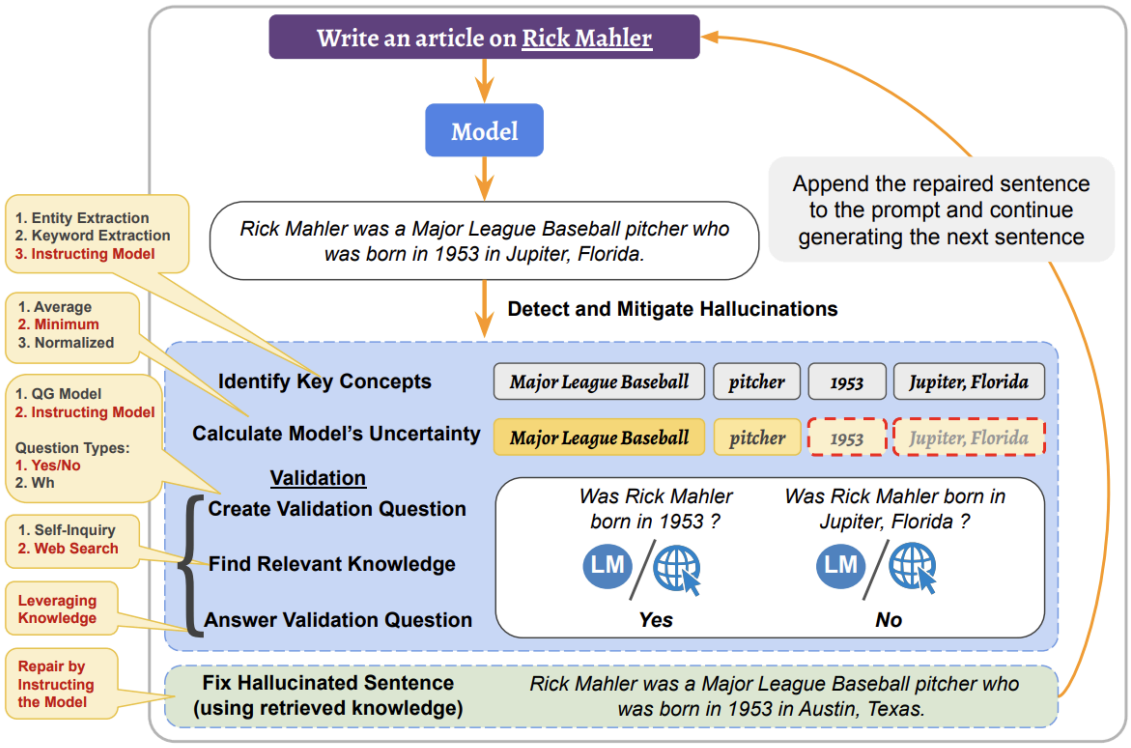
   – Factuality (фактологическая точность относительно эталона),

   – Semantic entropy (разброс смыслов, если повторять запросы 3–5 раз на разных сидах).
7. Автоматически сгенерируй отчет с показателями по каждому вопросу и модели.

Как это сделать:

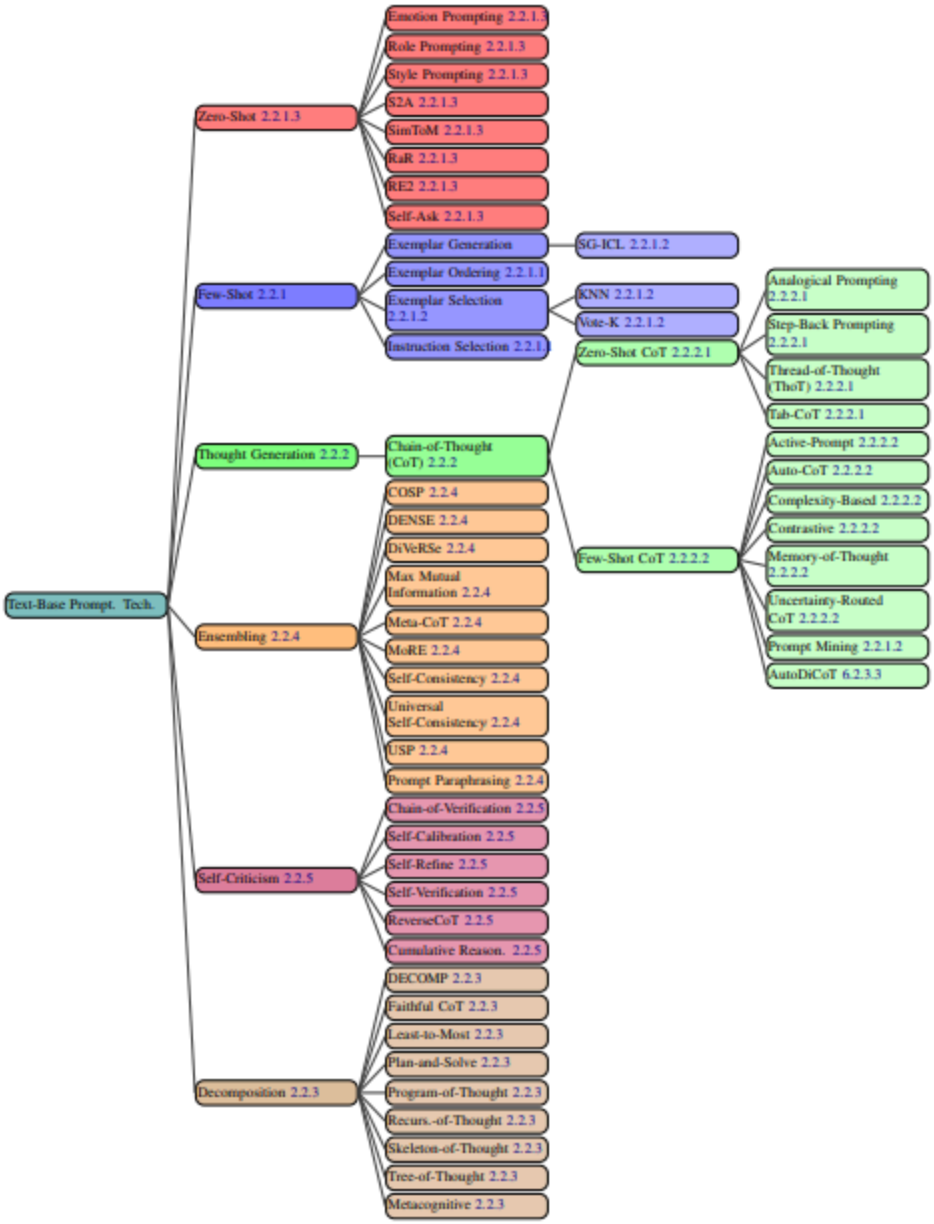
В Colab или Claude Artifact сам напиши код, подключи ключи, разберись, почему у тебя не работает импорты, получи ошибку, прочитай StackOverflow, перепроверь лимиты API, убей час жизни, а потом почувствуй себя настоящим инженером данных

IV. Что делать: инженерия и практики



10. Приёмы снижения галлюцинаций

10.1. Инженерия промптов



1. **ICE (Instruction, Constraints, Escalation)**
    - **Instruction**: чётко сформулируйте, что вы хотите получить (например, "дай только список стран").
    - **Constraints**: укажите ограничения — например, "используй только факты из 2020 года" или "не придумывай, если не знаешь".
    - **Escalation**: заложите сценарий «не знаю» — например, попросите явно отвечать “нет данных” или “не уверен”, если нет фактической

информации.

- Эта структура повышает прозрачность вывода и уменьшает количество выдуманных деталей.

2. **Step-Back Prompting**
    - Просим модель "остановиться и подумать": например, сначала попросить оценить свою уверенность или разложить задачу на шаги.
    - Такая техника снижает "импульсивные" галлюцинации, когда модель сразу выдаёт первое "угаданное" значение, не проверяя его внутренне.
    - Вариация: спросите модель, какие есть альтернативные версии ответа или какие данные нужны для проверки.
3. **Chain-of-Verification**
    - Встраиваем в промпт явные инструкции по верификации: сначала сгенерируй ответ, затем придумай вопросы для самопроверки, затем перепроверь их по имеющимся фактам.
    - Например: "Дай короткий ответ, затем сформулируй два уточняющих вопроса, которые помогут проверить твой ответ, и попробуй на них ответить".
    - Такой подход хорошо работает в задачах поиска информации и аналитики, снижая количество очевидных ошибок и упрощая аудит результата.
4. **Veracity Classification Prompts**
    - Добавьте в промпт шаг "оцени достоверность каждого факта" или "дай вероятность, что факт верный".
    - Позволяет быстро выявлять "слабые места" в сгенерированном ответе, хотя не всегда полностью устраняет галлюцинации, т.к. модель может быть уверена в неверном ответе.

Практика

Промпт без структуры:

«Объясни роль Scrum Master.»

Промпт:

«Перечисли только фактические обязанности Scrum Master в формате списка. Используй только проверенные источники (например, Scrum Guide). Если есть сомнения — напиши “нет данных” или “не уверен”.»

- *Промпт:*

  «Перед тем как ответить, оцени свою уверенность в знании темы (в процентах). После этого разложи роль Scrum Master на шаги: сначала опиши ключевые задачи, потом дай итоговое резюме.»

- *Промпт:*

  «Дай краткий ответ на вопрос “Что делает Scrum Master?”. Затем сформулируй два вопроса для самопроверки и попытайся ответить на них, используя только официальные материалы по Scrum.»

- *Промпт:*

  «Перечисли обязанности Scrum Master, а рядом с каждым пунктом укажи: (а) насколько ты уверен в его достоверности (в процентах), (б) укажи источник, если есть.»

---

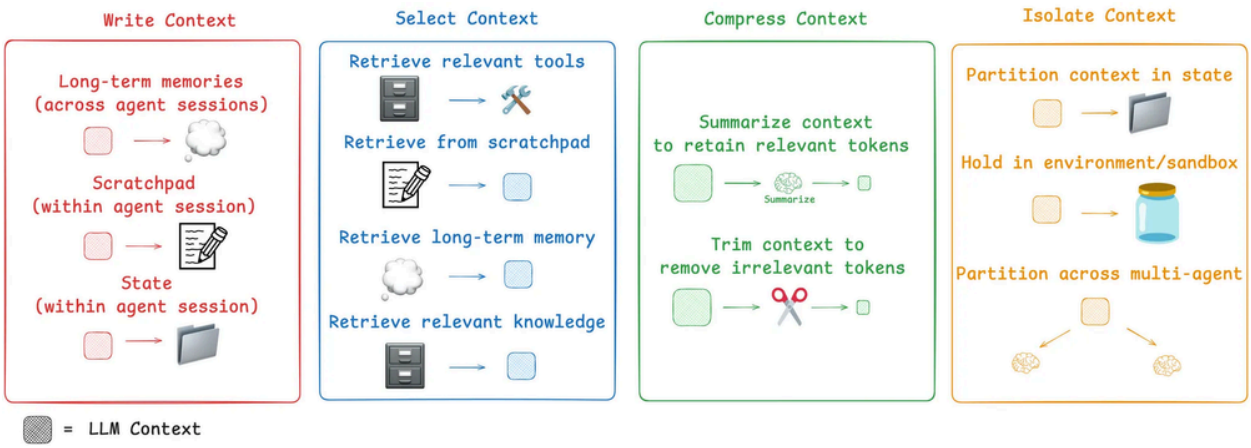
5. A/B/C/... Playground Test

1. Открой PromptHub/Poe или любой playground, где можно создавать пайпы с разными промптами.
2. Создай отдельные пайпы для каждого подхода (ICE, Step-Back, Chain-of-Verification, Veracity Classification).
3. Используй одну и ту же LLM (например, GPT-4, Claude, Gemini) для всех пайпов — чтобы сравнивать именно структуру промпта.
4. Прогоняй один и тот же вопрос через каждый пайп.
5. Оцени результаты:
   - Где ответ точнее и прозрачнее?
   - Где модель честнее признаёт “не знаю”?

- Где меньше “воды” и очевидных галлюцинаций?
- Сделай A/B/C сравнение для одного вопроса (ручной scoring или Google Forms с оценкой по шкале faithfulness/factuality).

6. Если playground поддерживает “мульти-запуск” (batch/pipe), прогони сразу несколько тем или вопросов.

10.2. Системные подходы



1. Инженерия контекста (Context Engineering)

Это не просто “правильная формулировка промпта”, а осознанное управление всей информацией, которую получает LLM: инструкции, ввод пользователя, история, внешние факты, ограничения. Контекст — это как “оперативная память” ИИ.

- **Фильтрация и структура** помогают не “захламлять” внимание модели: если контекст чётко разделён (например, `[ИНСТРУКЦИЯ]`, `[ИСТОРИЯ]`, `[ДАННЫЕ]`), модель реже путается между целями задачи и “додумывает”.
- **Иерархия важности** (важное впереди, шум убран) снижает риск “контекстного дрейфа” — когда модель начинает опираться на несущественные детали или старые сообщения.
- **Явные ограничения** (“не делай предположений, если данных нет — напиши ‘нет данных’”) заставляют модель признавать незнание, а не фантазировать.

В корпоративном ассистенте при запросе “кто одобряет командировки?” промпт включает: “Используй только политику из базы. Если в политике нет информации, напиши: ‘Информация отсутствует’.” Это снижает число галлюцинаций, когда ИИ мог бы “придумать” ответ, основываясь на шаблонах из общедоступных данных.

Ключевые нюансы:

- Контекст легко “засорить” старыми или противоречивыми кусками — поэтому регулярная чистка и структурирование обязательны для стабильной работы.
- **Контекстная изоляция**: если параллельно идут разные задачи — разделяйте их память, иначе модель начнёт смешивать инструкции и результаты (типичный источник “системных галлюцинаций”.

Практика

Соберите нодовый пайплайн в n8n, который автоматизирует:

1. **Извлечение контекста** по davidkimai/Context-Engineering (используем инструкции, примеры — templates).
2. **Формирование промпта** с несколькими блоками контекста (instructions + примеры + память пользователя).
3. **Запрос** к LLM через OpenAI или другой node.
4. **Возврат** ответа в Telegram-чат, где спрашивал пользователь.

Что получите:

- Написанный бота, автоматом подставляющего контекст в промпт.
- Видите сразу: без контекста — ответ хаотичный; с контекстом — структурированный, без “галлюцинаций”.

Почему это важно:

Поиск в `Context-Engineering` — это базовый шаблон «сборка контекста + LLM». Здесь вы на практике увидите, как строгая контекст-инъекция влияет на

качество ответов и отключает “липовую” генерацию.

Выполните:

1. Клонируйте репозиторий `Context-Engineering` и выберите пару примеров из `20_templates/`.
2. Сконфигурируйте n8n:
   - Telegram Trigger → HTTP Request (шаблон) → Set (собрать контекст) → LLM Node → Telegram Send.
3. Проверьте пару запросов с и без шаблонов.
4. Оптимизируйте порядок контекста, убедитесь, что получаете более точные и логичные ответы.

---

2. RAG (Retrieval-Augmented Generation)

Механизм, при котором LLM не просто генерирует ответ “из головы”, а подтягивает релевантные факты из базы (FAQ, документы, корпоративные регламенты, интернет).

Почему снижает галлюцинации:

- Галлюцинации часто появляются, когда модель вынуждена “догадываться” на неполном знании. RAG даёт ей “свежий” материал для ответа, минимизируя число домыслов.
- *LLM с RAG на 20–30% точнее отвечает на вопросы, где требуются факты, не попавшие в её обучение*. Но если поиск возвращает нерелевантное, ошибка переходит на следующий этап (важность фильтрации).

Сотрудник спрашивает про новую HR-политику. Ассистент с RAG обращается к свежим инструкциям в базе — и возвращает цитату с точной ссылкой, а не стандартный или устаревший ответ.

RAG не отменяет проверки: если в базе нет нужного, LLM может всё равно “придумать” ответ. Поэтому важно комбинировать RAG с инженерией контекста: “Отвечай только если нашёл релевантный документ, иначе — ‘нет данных’”.

Практика

1. **Возьмите профессиональную книгу** (например, Scrum Guide Expansion Pack 2025).
2. **Составьте 12 вопросов** по ключевым разделам книги.
3. **Подготовьте “золотой” бенчмарк**: для каждого вопроса выпишите эталонный (короткий и точный) правильный ответ, опираясь на сам текст книги.
4. **Задайте каждый вопрос обычной LLM** (например, DeepSeek или другую в playground) по 10 раз — итого получите 120 ответов.
5. **Сравните все ответы с бенчмарком**:

   – вручную (или с помощью simple script) оцените совпадения, неточности, галлюцинации;

   – посчитайте долю верных, неверных и “галлюцинаторных” ответов.
6. **Векторизуйте книгу и настройте RAG** (например, через HuggingFace/FAISS, Haystack или Pinecone, неважно — любой доступный пайплайн).
7. **Повторите те же 12 вопросов по 10 раз** уже через RAG — снова получите 120 ответов.
8. **Сравните метрики по бенчмарку для обычной LLM и для LLM+RAG**:

   – где выше точность, где меньше выдумки, где больше ссылок на оригинал.
9. **Сделайте выводы — в чем и насколько RAG улучшил реальную “фактологическую” устойчивость вашей LLM.**

---

3. Web search / Deep research base

Подключение к онлайн-поиску (новости, аналитика, научные базы). Это расширяет возможности RAG: ищем не только внутри компании, но и “во внешнем мире”.

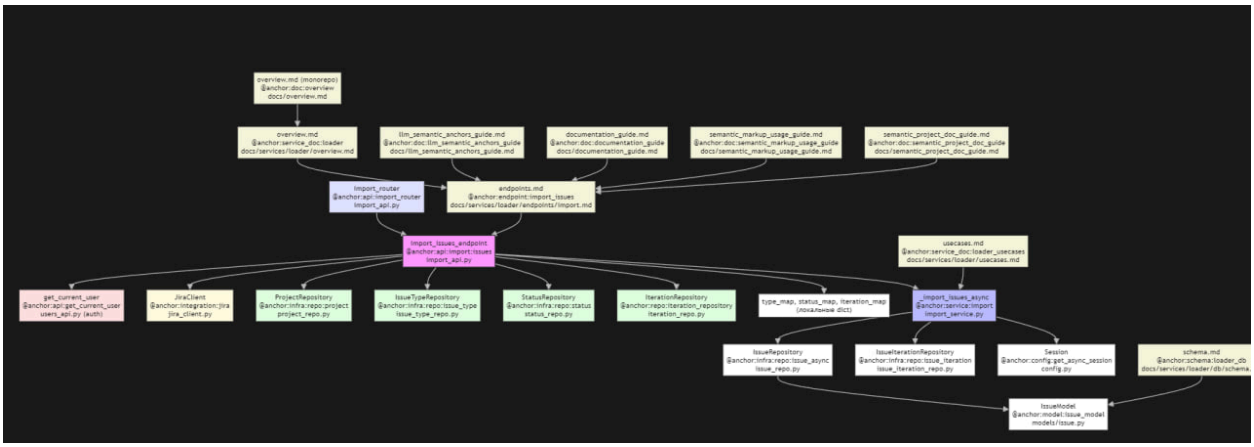
- Позволяет найти подтверждение факта или опровергнуть устаревшие сведения, если внутри компании данных нет.
- **Но!** Как показано в обзорах, без жёсткой фильтрации найденной информации AI может “подцепить” фейк, устаревшую новость или фрагмент, не относящийся к вопросу — и уверенно её сгенерировать.

AI-ассистент анализирует новости конкурентов. Он подтягивает только те статьи, где упоминается конкретная компания и фильтрует по дате публикации, чтобы не ссылаться на старую информацию.

Web search — полезен для “белых пятен” (новое, уникальное), но требует валидации: либо ручной проверки источника, либо автоматического “гварда” на факты.

---

4. Семантические графы (Knowledge Graphs)



Использование структур знаний (связи между людьми, процессами, документами), чтобы AI делал выводы на основе реальных связей, а не только на статистике текста.

- Модель опирается на конкретные связи, что снижает риск “ошибочных сопоставлений” (например, не перепутает похожие термины или отделы).
- Внедрение графов особенно эффективно в сложных, многосвязных бизнес-процессах — уровень ошибок при поиске информации снижается, ответы становятся повторяемыми и обоснованными.

Менеджер спрашивает: “Кто отвечает за направление X?”. AI достаёт связь из графа “Проект → Руководитель” и возвращает не предположение, а реальную структурную единицу.

---

5. Domain-specific knowledge → LoRA / Fine-tuning

Лёгкое дообучение на своей базе кейсов (LoRA), либо полноценное дообучение модели на профессиональных данных.

- Уменьшает ошибки и “фантазию” в терминологии, редких ситуациях и внутренних процедурах.
- В исследованиях (Ji et al. 2023, Huang et al. 2024) доменное дообучение снижает количество галлюцинаций и повышает вероятность честного “не знаю” вместо излишней уверенности в неправильном ответе.

После дообучения на документах техподдержки AI перестаёт предлагать несуществующие способы решения, а использует только реальные инструкции компании.

Ключевые нюансы:

- Критична “чистота” и актуальность данных для обучения: иначе новая ошибка становится системной.
- Лучше использовать регулярное обновление и ревизию базы знаний.

---

6. Guardrails / Output validation

Многоуровневые проверки на этапе выдачи ответа. Это фильтры, повторные вопросы, сравнение с эталонными ответами или автоматический мета-анализ (SelfCheckGPT, Chain-of-Verification, LLM-Check).

- Ловит “выдумки” на последнем этапе: если даже предыдущие фильтры пропустили ошибку, guardrail проверит ответ повторно или сверит с источниками.
- В [LLM-Check, 2023] показано, что такие фильтры снижают число критических галлюцинаций до 2–5% (против 10–15% без них).

AI генерирует отчёт, и после этого специальный модуль задаёт ему уточняющий вопрос: “Подтверди все факты ссылками на источник”. Если

подтверждение не найдено — менеджер получает предупреждение о возможной галлюцинации.

Можно комбинировать “мягкие” фильтры (уведомить пользователя о сомнительном факте) и “жёсткие” (блокировать ответ или требовать дополнительного подтверждения).

10.3. Житейские подходы

Перекрёстная проверка: «спроси ещё одну модель»

Один из самых простых и эффективных приёмов — проверить сомнительный ответ LLM через другой сервис или «дать ответу пройти тест на суровую реальность». Например: получили ответ от ChatGPT — задайте тот же вопрос Gemini, Claude или даже более старой версии ChatGPT. Такой способ называется LLM-as-a-judge или cross-LLM validation.

- Исследования показывают, что модели часто галлюцинируют независимо друг от друга, но вероятность одинаковой ошибки у нескольких моделей ниже.
- Если 2–3 модели выдали одно и то же (или почти совпадающее) по смыслу, скорее всего, информация верна.
- Если разброс большой — это сигнал насторожиться и пойти искать подтверждение в внешних источниках или ручками в Google

Минусы:

- Некоторые LLM могут «копировать» популярные ошибки друг у друга, особенно на трендовых данных или на одних и тех же датасетах.
- Метод не работает на редких вопросах, где моделей просто нечему «сверяться» (нет консенсуса).

Критическое мышление: «поверь, но проверь»

Наконец, любой автоматический инструмент — не замена человеческому скепсису. Исследования и рекомендации крупных компаний (Microsoft, OpenAI, Red Hat) советуют всегда проверять ответы LLM с позиции: «Если бы мне это сказал стажёр — я бы поверил?» Или: «Смогу ли я подтвердить это в авторитетных источниках?»

Практические советы:

- Не принимайте на веру факты, особенно если они необычные, важные для бизнеса или принимаются для принятия решений.
- Привычка «гуглить» непонятные факты и сверяться с первоисточниками работает не хуже, чем многие автоматические проверки.
- В высокорисковых сценариях (медицина, финансы, юриспруденция) — только с верификацией через профильных специалистов или документацию.

10.4. Перспективные подходы

В последние годы одними из самых перспективных направлений в борьбе с галлюцинациями LLM считаются интеграция языковых моделей с внешними источниками знаний (например, knowledge graphs — графы знаний) и развитие подходов "прозрачного" ИИ, где модель прямо показывает, какие источники и инструменты использует для вывода. Пример: Toolformer — архитектура, в которой LLM учится использовать внешние инструменты и API для уточнения фактов или выполнения операций.

Традиционные LLM работают как “черный ящик”: они генерируют текст на основе статистики из обучающих данных, не объясняя свои решения и не ссылаясь на источники. Это приводит к двум ключевым проблемам:

- Пользователь не понимает, насколько ответ основан на реальных фактах.
- Даже если модель уверена, она может “галлюцинировать” — выдавать несуществующую или ошибочную информацию с очень убедительным видом.

Классическая проблема: LLM часто "выдумывает" факты, которых нет ни в обучении, ни в реальности. Но если модель получает возможность запрашивать, интегрировать и ссылаться на knowledge graph (структурированная база данных фактов) или называть используемые внешние инструменты, вероятность галлюцинаций резко падает:

- Модель проверяет и подставляет реальные факты из базы (например, Wikidata или корпоративного справочника).

- Прозрачная цепочка рассуждений позволяет понять, когда LLM “перепутала” источник или домыслила.

Куда развивается рынок и исследования

- **Тенденция**: всё больше новых LLM (особенно в B2B-продуктах) “умеют” работать с внешними базами, проверять факты на лету и объяснять свои действия — это стандарт де-факто в современных корпоративных решениях.
- **Горячие темы**: построение “explainable AI”, генерация ссылок на источник в реальном времени, аудит reasoning chain, развитие инструментальных LLM (Toolformer, AutoGPT и т.д.).

Вывод

Для менеджеров и практиков ключевой урок: если вы строите сервис с LLM и хотите минимизировать галлюцинации, фокусируйтесь на интеграции с проверяемыми источниками знаний и прозрачности reasoning. Даже лучшие языковые модели не гарантируют стопроцентной точности “из коробки”, но современные “knowledge-augmented” и “tool-using” подходы — мощный инструмент против галлюцинаций.

11. Что **не** помогает

Вокруг LLM сложился целый арсенал “магических приёмов”, которые вроде бы должны снижать количество галлюцинаций. Однако, по итогам исследований и обзоров, большинство из них на деле оказываются малоэффективными — если не переобучать модель или не использовать дополнительные источники фактов.

“According to...”

Формулировки вроде “According to Wikipedia...” или “Ссылайся только на проверенные источники” в промпте создают иллюзию контроля, но не дают реального улучшения достоверности. Модель продолжает генерировать текст на основе своих внутренних вероятностей, а не реального доступа к внешним базам данных или онлайновым источникам. В систематическом обзоре Ji et al. (2023) и последующих работах подчеркивается: LLM не

“ходит” за фактами, она просто подражает стилю цитирования, даже если этих фактов нет в корпусе данных.

Negative prompting

Попытки отговорить модель от ошибок с помощью “негативных” инструкций (например, “Не придумывай”, “Не выдумывай” и т.д.) — почти не работают. Как показано в обзорах и экспериментах, модель не понимает эти ограничения буквально и всё равно может “галлюцинировать”, если у неё нет нужных знаний или она не уверена в ответе.

Назначение ролей (“Ты эксперт по...”)

Инструкция “Представь, что ты врач/юрист/аналитик...” меняет стиль и тон ответа, но не достоверность содержимого. Модель лучше маскирует уверенность, но под капотом работает та же статистическая “угадайка”, которая не становится более фактической или “экспертной” от этой роли. Это отмечается во многих обзорах: экспертная роль влияет на формулировки, но не на фактологическую точность.

Reflective prompting

Приёмы типа “Подумай ещё раз”, “Проверь свой ответ”, “Объясни, почему ты так думаешь” (reflective/self-reflective prompting) действительно помогают чуть чаще ловить внутренние противоречия, но без внешнего знания или дополнительного обучения почти не снижают реальное число фактологических ошибок. Большинство моделей склонны быть “уверенными в своих ошибках” и повторяют их с разной формулировкой, а не признают незнание.

Простое “Say I don’t know” без переобучения

Добавление инструкции “Если не знаешь — скажи, что не знаешь” работает только при дообучении на специальных датасетах или внедрении внешних гвардов (guardrails). Обычные LLM “из коробки” по умолчанию редко используют фразу “не знаю”, потому что это противоречит основной задаче генерации — выдавать наиболее вероятный следующий токен, а не “признавать незнание